CampusEats Design Benchmark

Services, Contracts & Database Schema Design

Task 1: System Capabilities (Verbs)

*** RegisterUser**

Creates a new profile for a Customer, Rider, or Merchant, sets credentials, and logs identity metadata. (Owned by User Service)

*** BrowseMenu**

Fetches active canteens, available menu categories, and item catalog pricing parameters. (Owned by Canteen Service)

*** PlaceOrder**

Creates an order ticket, aggregates cart line items, calculates total prices, and handles transaction status. (Owned by Order Service)

*** ProcessPayment**

Verifies funds, interfaces with campus/digital payment gateways, and logs transaction audits. (Owned by Payment Service)

*** AssignRider**

Coordinates logistics, identifies idle riders on campus, dispatches routes, and creates delivery tickets. (Owned by Delivery Service)

*** UpdatePrepStatus**

Allows canteen merchants to update order preparation steps (Cooking, Ready for Pickup) in real-time. (Owned by Order Service)

Task 3: Service API Contracts

1. User Service (Lead: Nandini)

Operation	Inputs	Outputs	Errors
registerUser	name, email, phone, user_type, password	user_id, created_at	EMAIL_ALREADY_EXISTS, INVALID_PHONE
authenticateUser	email, password	session_token, user_id, role	CREDENTIALS_INVALID, USER_LOCKED
getUserProfile	user_id	name, email, phone, role	USER_NOT_FOUND

2. Order Service (Lead: Nandini)

Operation	Inputs	Outputs	Errors
placeOrder	customer_id, canteen_id, items_list, dropoff_loc	order_id, status, total_price, est_time	CANTEEN_CLOSED, ITEM_OUT_OF_STOCK, PAYMENT_DECLINED
getOrderStatus	order_id	order_id, status,	ORDER_NOT_FOUND
cancelOrder	order_id, customer_id	cancelled_at, status, refund_id	ORDER_NOT_FOUND, PREP_ALREADY_STARTED

Task 3: Service API Contracts (Continued)

3. Canteen (Catalogue) Service (Lead: Nandini)

Operation	Inputs	Outputs	Errors
getCanteenCatalog	canteen_id	canteen_details, menu_items_list	CANTEEN_NOT_FOUND
updateItemAvailability	menu_item_id, is_available	updated_status, success	ITEM_NOT_FOUND, UNAUTHORIZED

4. Delivery Service (Lead: Alka)

Operation	Inputs	Outputs	Errors
scheduleDeliveryJob	order_id, canteen_id, dropoff_building, fee	job_id, initial_status	INVALID_BUILDING, ORDER_ALREADY_SCHEDULED
updateRiderLocation	rider_id, lat, lng	ack, status	RIDER_NOT_FOUND

5. Payment Service (Lead: Alka)

Operation	Inputs	Outputs	Errors
chargeCard	order_id, user_id, amount, pay_method	txn_id, status, timestamp	INSUFFICIENT_FUNDS, GATEWAY_TIMEOUT
refundPayment	txn_id, amount	refund_txn_id, status	TRANSACTION_NOT_FOUND, ALREADY_REFUNDED

Task 4: Specification of placeOrder Operation

1. Operation Signature & Interfaces

Name: placeOrder

Interface Layer: REST Endpoint / HTTP POST (/api/v1/orders)

Inputs:

- customer_id: UUID (Unique identifier of the authenticated customer)
- canteen_id: UUID (Unique identifier of the source campus canteen)
- items: List of objects, each containing:
  * menu_item_id: UUID (Unique identifier of the ordered food item)
  * quantity: Integer (Must be greater than 0)
  * customization_notes: String (Optional, e.g., 'no onions')
- dropoff_building: String (Valid target campus building name, e.g., 'Science Lab A')

Outputs:

- order_id: UUID (Unique ID generated for the accepted order)
- order_status: String (Will be initialized to 'placed')
- total_price: Decimal (Total invoice cost including subtotal, tax, and delivery fees)
- estimated_delivery_time: Timestamp (Calculated arrival timeframe)

2. Error Cases & Business Rules

- CUSTOMER_NOT_FOUND (404): The user ID provided does not match any registered customer account.
- CANTEEN_CLOSED (400): Canteen has toggled operating status to offline or request timestamp falls outside business hours.
- ITEM_OUT_OF_STOCK (400): One or more ordered menu items is flagged as unavailable.
- INVALID_BUILDING (400): The dropoff building string is not part of the active geofenced campus database.
- PAYMENT_DECLINED (402): Transaction processing failed due to insufficient funds on the campus card or gateway error.

3. Hidden Internal Details ('Hiding the Inside')

To enforce loose coupling, the placeOrder contract hides all implementations of how the order is fulfilled:

1. Schema Independence: Callers do not know that placing an order updates two separate PostgreSQL tables ('orders' and 'order_items') under the hood.
2. Service Orchestration: The Order Service coordinates the flow by fetching authentication status from the User Service and validating prices against the Canteen Service asynchronously. This microservice choreography is transparent to the client.
3. Payment Integration: The actual billing mechanisms (interfacing with payment gateways via HTTP API or debiting tables) are isolated within the Payment Service.
4. Dispatch Queues: The placeOrder operation inserts a message into an asynchronous rider dispatch queue (Redis streams) to trigger a matching process in the Delivery Service, decoupling order taking from delivery execution.

Task 6: Service Property Validation

~~We validate our 5 services against the core properties of a true Service: Reachable, Self-contained, Has a Contract,~~ Independent, and Loosely Coupled. Each service meets the criteria below:

Service	Property Status & Verification Check	Remediation / Corrective Actions
User Service	Reachable: Yes (Exposes standard HTTP REST APIs).	No defects found. Completely self-contained.
Canteen Service	Reachable: Yes (Catalog API). Self-contained: Yes (Owns users / user_credentials tables).	No defects found. Completely self-contained.
Order Service	[illegible] [illegible] [illegible] Coupled: Low / Independent: Yes (Exposes catalog without order details). Coupled: Moderate (Requires active menu data to validate prices).	To maintain strict loose coupling during network latency, Order Service should cache menu items locally instead of making blocking synchronous calls to the Canteen Service during checkout.
Delivery Service	Reachable: Yes (scheduleDelivery API). Self-contained: Yes (Owns rider_profiles & delivery_jobs).	No defects found. rider_id and order_id are mapped logically to keep separation.
Payment Service	Reachable: Yes (chargeCard API). Contract: Yes (scheduleDelivery/updateRiderLocation). Independent: Yes. Self-contained: Yes (Owns payments & refunds tables). Coupled: Low / Contract: Yes (Decoupled from order_id/amount details via logical order_id mapping). [illegible] (Inputs order_id/amount, Outputs status/txn_id) Independent: Yes (Integrates outward gateway connections). Coupled: Low (Exposed via logical order_id).	No defects found. Isulates billing away from core order processing.